Learning Objectives

This lab lets you practice writing your own class and implementing several functions for it.

Estimated Size

1 program file: `thermostat.py`, 4 functions.

Setup

In your Python files, you must include author information. On the first line, add a `# Author` comment line, where you mention your full name. Similarly, include your email and Spire ID on the `# Email` and `# Spire ID` comment lines, respectively. Remember, these comments must appear **at the beginning of every Python file** that you submit to Gradescope. For example:

```
# Author   : John Snow
# Email    : jsnow@umass.edu
# Spire ID : 12345678
```

0. Programmable Thermostat

A programmable thermostat allows you to adjust room temperatures according to a predefined set of schedules. A schedule is defined by a time-temperature pair, where the time is a string (e.g. `'08:15'`) and the temperature is a float (degree in Fahrenheit). For example, the set of schedules may look like: `'05:45'→68 degrees, '09:00'→60 degrees, '17:25'→68 degrees, '22:00'→70 degrees`

For simplicity, we represent a time in 24-hour format by a string of exactly 5 characters, where the minute and second are 0-padded (i.e. if the value is less than 10, it's padded with a leading 0). This **allows us to compare times simply by using string's comparisons**. In addition, the thermostat will have a default temperature value, so in the case there is no preset schedule, it will maintain the room at the default temperature value. In the following you will write a class called `Thermostat` to allow a user to add schedules and later query the schedules to find the target temperature given an arbitrary time.

1. Implement the constructor function (2 points)

Create a file called `thermostat.py`, and in the file create a class called `Thermostat`. Your first task is to implement its constructor function: other than the `self` parameter, it should take a number value as the default temperature parameter -- this parameter should be given a **default value of 68**, so in the case the user doesn't provide this parameter, the default temperature will be 68.

In the constructor you should create two instance variables / attributes / properties: one to store the default temperature, and another to store the schedules like described above. The most straightforward data type for schedules is a dictionary, as you can use it to store time-temperature pairs. So in the constructor, you should assign it to an empty dictionary indicating there is no schedule yet. Once you have the constructor implemented, you can create objects of the `Thermostat` class by using, for instance: `Thermostat()` or `dev = Thermostat(75)`

Hint: Do **NOT** ask the user for any input in the function. You **should not** take schedule as a parameter, and should instead make a new empty dictionary for every thermostat object. If you do not, this can lead to a strange bug in the __str__ function

2. Implement function `add_schedule` (2 points)

Your next task is to implement a function for this class called `add_schedule`. Other than the `self` parameter, this function should take a time (string) and a temperature (float) as parameters. It should store the time-temperature pair (where time is the key and temperature is the value) in the schedules dictionary. If the time already exists, its value will be overwritten by the new temperature; otherwise it will create a new entry. You can assume the time string always represents a valid time. After this function is implemented, you can add schedules to your `Thermostat` object by for instance:

```
dev = Thermostat(75)
dev.add_schedule('08:00', 60.4)
dev.add_schedule('19:35', 75.5)
dev.add_schedule('09:30', 58.2)
dev.add_schedule('22:39', 68.2)
```

Note that the user doesn't need to add schedules in increasing/ascending order of time -- the schedules can be added in any arbitrary ordering of times.

Hint: Do **NOT** ask the user for any input in the function.

3. Implement the cast-to-string function `__str__` (5 points)

Now you will implement the special function `__str__` which casts this object to a string. Specifically, this function should **return a string** (note: return, NOT print) formatted as the following:

```
Default temperature: {default_temp} degrees
{TIME1} {VALUE1} degrees
{TIME2} {VALUE2} degrees
... ...
```

where `{default_temp}` is the actual value of the default temperature, and following from that are the time-temperature pairs **sorted in increasing/ascending order of time**, one on each line. Because the string should contain multiple lines, at the end of each line there must be a newline character `'\n'`, **except** the last line, which should NOT have a trailing newline character.

You don't need to implement the sorting algorithm yourself: Python provides a `sorted()` function, which can take a dictionary and returns **a list that contains the keys sorted in ascending order**. For example, in our case, `sorted(self.schedules)` returns a list of times sorted in ascending order.

When you are done, test this function by calling `print(Thermostat())`, it should print out just one line:
`Default temperature: 68`
which is what it should look like when no schedule is in the thermostat yet. Then, call `print(dev)` where `dev` has 4 schedules as seen in Section 2 above. This should print out 5 lines:

```
Default temperature: 75 degrees
08:00 60.4 degrees
09:30 58.2 degrees
19:35 75.5 degrees
22:39 68.2 degrees
```

Hint: Do **NOT** ask the user for any input in the function. If you are struggling, try not to do it all in one line, we need to access every value in schedule using the key, this requires us to use a for-loop (or a comprehension).

When aggregating values to a string, think about how you would solve this problem if you were supposed to return a list. You would likely make an empty list, then add values to the end of it as you create them. You can do that same process but with a string instead.

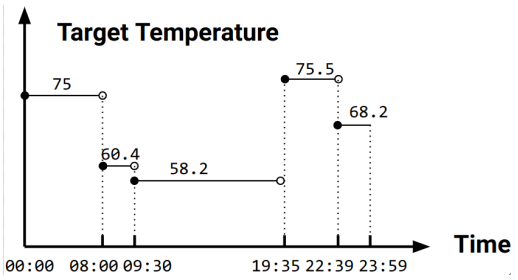
If you are getting a strange error in the autograder, where it looks like the autograder was expected one line (just the default temperature with no schedule) **but** you are returning a schedule, check your solution to part 1, and the ***hint*** to that part.

4. Implement function `get_target_temperature` (6 points)

Your last task is to implement a function called `get_target_temperature`. It should take an arbitrary query time (string) as the parameter, and it should **return** the target temperature at that query time. Imagine we **sort** the schedules in **ascending order of time**: `TIME1, TIME2, TIME3`, and so on. This divides the entire timeline of a day `00:00 to 23:59` into intervals. If a query time falls in `[TIME1, TIME2)` i.e. `TIME1` inclusive and `TIME2` exclusive, the target temperature should be that corresponding to `TIME1`; similarly if a query time falls in `[TIME3, TIME4)` the target temperature should be that of `TIME3`, and so on. If the query time is earlier than `TIME1` (i.e. the earliest time in `schedules`) this function should return the default temperature value. If the query time is after the last time, it should return the temperature set at that last time.

For example, given the `dev` object in Section 2 above (which has 4 schedules), the image on the right below shows a visualization of how the target temperature changes over a day according to the schedules. The table on the left below shows some sample query times and expected return values of calling this function.

`dev.get_target_temperature('23:00')`	`returns 68.2`
`dev.get_target_temperature('16:35')`	`returns 58.2`
`dev.get_target_temperature('05:55')`	`returns 75`
`dev.get_target_temperature('08:00')`	`returns 60.4`
`dev.get_target_temperature('12:00')`	`returns 58.2`



Hint: there are many ways to implement this function. One possible solution is to find the **latest/largest time** in `schedules` that is **smaller/earlier than or equal to** the query time, and return the temperature value corresponding to that schedule time. If you can not find any time in `schedules` that is smaller than or equal to the query time, it should return the default temperature value.

Finally we would like to reiterate that if you encounter any trouble with your code, please get used to **leveraging the debugging tool** to help you figure out the problem. Unlike writing an essay, where you may find mistakes/typos/syntax errors by repeatedly reading the text -- when you write a complex program, it's nearly impossible to eyeball bugs, less to say fix them, without running the code line by line in the debugger to examine the variable values. We would like you to form a habit to use the debugging tool, which is hugely beneficial as you go into upper-level programming classes.

Hint: Do **NOT** ask the user for any input in the function. The first approach you happen upon to solve the problem might not be the easiest, if you get stuck trying thinking about it in the **other direction**.

(Common Paragraph) Submission to Gradescope

Log in to Gradescope, select **Lab 10 (Code)**, and submit the required file.

Also remember to submit the lab questionnaire. To do so, open the **Lab 10 (Questionnaire)** Google Doc link (available on Piazza), make a copy so you can edit the file in Google Doc. When you are done, download it as a PDF and submit it to Gradescope. Select all your group members when submitting.

(Common Paragraph) Academic Honesty

All work that is completed in this assignment is your own (if you work individually) or your own group's (if you work in a group). You may talk to other students about the problems, and brainstorm about solutions. However, you may NOT share code in any way, except with your group members. What you submit **must be your own or your own group's work**.

You may not use any code that is posted on the internet. If you are not sure it is in your best interest to contact the course staff. We will be using software that will compare your code to other students in the course as well as online resources. It is very easy for us to detect similar submissions and will result in a failure for the exercise or possibly a failure for the course. Please, do not do this. It is important to be academically honest and submit your work only. Please review the UMass Academic Honesty Policy and Procedures so you are aware of what this means.

Copying solutions, whether partial or whole, obtained from other students or elsewhere, is academic dishonesty. Do not share your code with your classmates, and do not use your classmates' code. If you are confused about what constitutes academic dishonesty you should re-read the course policies. We assume you have read the course policies in detail and by submitting this project you have provided your virtual signature in agreement with these policies.